HAFTA 4 • PAZAR

Generatif Modeller

VAE ile MNIST & DCGAN ile Fashion-MNIST

Encoder · Decoder · Latent Space · Generator · Discriminator · Adversarial Training

$z \in \mathbb{R}^2$	ELBO	64×64	4 Dosya
VAE Latent Boyut	VAE Kayıp Fonksiyonu	DCGAN Çıktı Boyutu	Python Uygulaması

09:00 — 17:00 · Teoriden pratiğe · CPU ile çalışır

Günün Planı — Hafta 4 Pazar

09:00–09:30 Geçen Hafta Özeti & Giriş	09:30–10:45 Generatif Modellere Giriş	10:45–11:00 Kısa Mola	11:00–12:00 VAE Matematik	12:00–13:00 VAE ile MNIST
13:00–14:00 Öğle Arası	14:00–15:30 GAN Teorisi & DCGAN	15:30–16:30 DCGAN ile Fashion-MNIST	16:30–17:00 Ödev & Kapanış	

Gün Sonu Hedefi: VAE ile MNIST latent space görselleştir → DCGAN ile Fashion-MNIST görüntüsü üret → eğitim sürecini analiz et

Generatif Modellere Giriş

Diskriminatif vs Generatif · Yoğunluk Tahmini · Veri Dağılımı Öğrenimi

Diskriminatif Modeller

P(y | x) — Sınır öğrenir

Giriş verisine bakarak hangi sınıfa ait olduğuna karar verir. Veri nasıl üretildiğini bilmez.

Örnekler:
- ResNet → kedi mi, köpek mi?
- BERT → pozitif mi, negatif mi?
- DistilBERT IMDB fine-tuning

Generatif Modeller

P(x) veya P(x | z) — Veri üretir

Verinin altında yatan dağılımı öğrenir. Yeni, gerçekçi veri örnekleri üretebilir.

Örnekler:
- VAE → yeni rakam görüntüsü üretir
- GAN → sahte yüz fotoğrafı üretir
- Difüzyon → metin→görüntü üretimi

VAE

Variational Autoencoder
Latent space: sürekli, yapılandırılmış
Reparameterization trick

GAN

Generative Adversarial Net
Generator vs Discriminator
Adversarial oyun dengesi

Difüzyon

Denoising Diffusion
Gürültü ekle → geri al
Stable Diffusion temeli

Flow

Normalizing Flow
Tersine çevrilebilir dönüşümler
Kesin yoğunluk hesabı

Bu hafta: VAE (MNIST) + DCGAN (Fashion-MNIST) · Hafta 5: Stable Diffusion, LLM deployment

VAE — Variational Autoencoder Matematiği

ELBO · KL Divergence · Reparameterization Trick · Latent Space

ELBO Kayıp Fonksiyonu

ELBO = E[log P(x|z)] − KL(Q(z|x) || P(z))

① Reconstruction Loss: orijinal görüntüyü ne kadar iyi yeniden oluşturabilir?
② KL Divergence: latent dağılım ne kadar N(0,I)'ya yakın?

Reparameterization Trick

Problem: $z \sim N(\mu, \sigma^2)$ → örnekleme geri-yayılım eğimi kesmez

Çözüm: $z = \mu + \sigma \cdot \varepsilon$, $\varepsilon \sim N(0,1)$

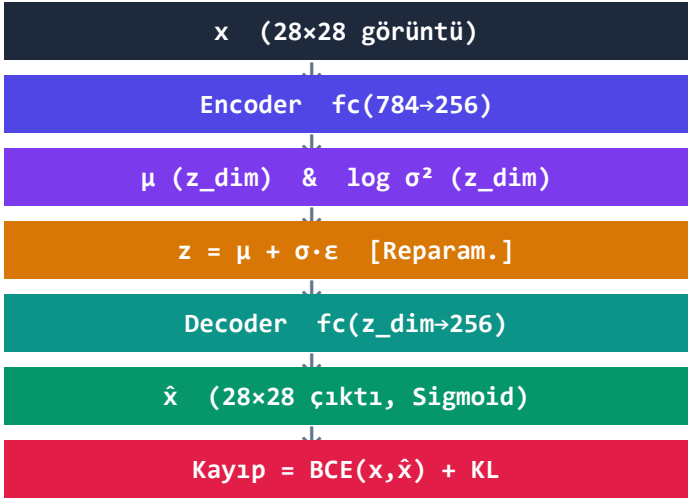
Gradyan μ ve σ üzerinden akar ✓
Örnekleme artık deterministik ✓

Encoder & Decoder

Encoder q(z|x): x → μ, log σ²
- Girdi: 28×28=784 boyut
- Çıktı: z_dim boyutlu (μ, σ)

Decoder p(x|z): z → x̂
- Girdi: z_dim boyut
- Çıktı: 28×28=784 boyut (sigmoid)

VAE Mimari Akışı



Latent boyut z_dim=2 → görselleştirme için ideal · Pratikte z_dim=64~256 kullanılır · Daha büyük z_dim → daha keskin görüntü

VAE — Keras ile MNIST Uygulaması

Encoder · Reparameterization · Decoder · ELBO Kaybı · Latent Space Görselleştirme

vae_mnist.py

```
import tensorflow as tf
from tensorflow import keras
import numpy as np

# ── Sampling Layer (Reparameterization) ──────────────
class Sampling(keras.layers.Layer):
    def call(self, inputs):
        z_mean, z_log_var = inputs
        eps = tf.random.normal(tf.shape(z_mean))  # ε~N(0,1)
        return z_mean + tf.exp(0.5*z_log_var) * eps

# ── Encoder ──────────────────────────────────────────
latent_dim = 2          # 2D latent: görselleştirme için
inputs = keras.Input(shape=(784,))
x = keras.layers.Dense(256, activation='relu')(inputs)
z_mean    = keras.layers.Dense(latent_dim)(x)
z_log_var = keras.layers.Dense(latent_dim)(x)
z = Sampling()([z_mean, z_log_var])
encoder = keras.Model(inputs, [z_mean, z_log_var, z])

# ── Decoder ──────────────────────────────────────────
z_inputs = keras.Input(shape=(latent_dim,))
x = keras.layers.Dense(256, activation='relu')(z_inputs)
outputs = keras.layers.Dense(784, activation='sigmoid')(x)
decoder = keras.Model(z_inputs, outputs)
```

```
vae.compile(optimizer='adam')  ·  vae.fit(x_train, epochs=30, batch_size=128)  ·  latent_dim=2 → 30 epoch → ~3 dk CPU
```

```
# ── ELBO Kaybı ───────────────────────────────────────
class VAE(keras.Model):
```

Latent Space Özellikleri

z_dim=2 → 2D düzlemde görselleştirme
Rakamlar küme oluşturur (0-9 renkleri)
Ara noktalar: rakam geçişleri
Örnekleme: N(0,I)'dan z çek → decode

Eğitim Parametreleri

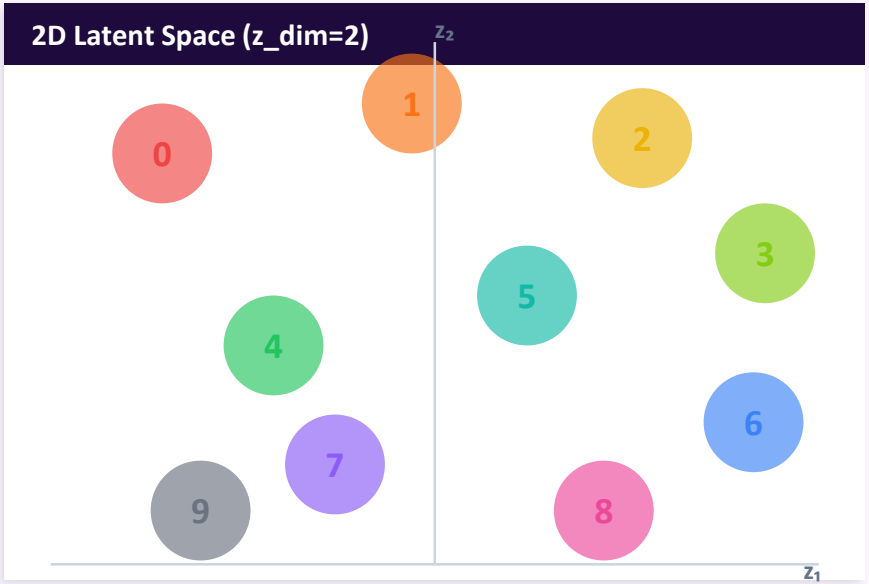
Veri : MNIST (60K train, 10K test)
Epoch : 30
Batch : 128
Optimizer: Adam(lr=1e-3)
Kayıp : ELBO = BCE + KL

Çıktı Görselleri

① 2D Latent Space: z uzayında rakam dağılımı (renk=sınıf)
② Manifold: 20×20 grid, -3 ile +3 arası
③ Eğitim kaybı (recon + KL)
④ Orijinal vs yeniden oluşturulmuş

VAE Latent Space Analizi — Neden Önemli?

Interpolasyon · Manifold · Anlamlı Boyutlar · Sınıf Kümeleşmesi



Sürekli Yapı

Birbirine benzer rakamlar (3-8, 1-7) latent uzayda yakın konumlanır. Ara noktalar anlamlı görüntüler üretir.

Manifold Örnekleme

20×20 grid: $z_1 \in [-3,3]$, $z_2 \in [-3,3]$. Her noktadan decode → görüntü. Sınırlar arası pürüzsüz geçiş.

Posterior Collapse

Sorun: KL terimi baskın olunca z görmezden gelinir. Çözüm: KL annealing (β başlangıçta 0 → 1).

Disentanglement

β-VAE (β>1): daha bağımsız latent boyutlar. Her z_1 kalınlık, z_2 eğim gibi tek bir özelliği kontrol eder.

Latent space interpolasyon: $z = \alpha \cdot z_1 + (1-\alpha) \cdot z_2$ $(\alpha \in [0,1])$ · İyi VAE: farklı sınıflar iyi ayrışır, geçişler pürüzsüzdür

GAN — Generative Adversarial Networks

Goodfellow et al. 2014 · Generator vs Discriminator · Minimax Oyunu · Nash Dengesi

$$\min_G \max_D V(D,G) = \mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log(1 - D(G(z)))]$$

Generator G

Amacı: D'yi kandırmak — sahte görüntüler üretir.
Girdi: z ~ N(0,I) (rastgele gürültü)
Kayıp: −log D(G(z)) (D'nin 1 demesini ister)

Eğitim adımları:
1. z ~ N(0,I) örnekle
2. G(z) ile sahte görüntü üret
3. D(G(z)) hesapla
4. −log D(G(z)) minimize et
5. Yalnızca G ağırlıkları güncellenir

İdeal: D(G(z)) = 0.5 (D kararsız)

Discriminator D

Amacı: Gerçek ↔ Sahte ayırt etmek.
Girdi: gerçek x veya sahte G(z)
Çıktı: P(gerçek) ∈ [0, 1]
Kayıp: BCE(D(x),1) + BCE(D(G(z)),0)

Eğitim adımları:
1. Gerçek batch: x ~ P_data
2. Sahte batch: z ~ N → G(z)
3. BCE(D(x),1) + BCE(D(G(z)),0)
4. Yalnızca D ağırlıkları güncellenir
5. Sonra G sırası

Mode Collapse: G tek tip örnek üretir · Vanishing Gradient: D çok iyi olunca G öğrenemez · Çözüm: DCGAN, WGAN, StyleGAN

DCGAN — Deep Convolutional GAN

Radford et al. 2015 · ConvTranspose2D · BatchNorm · LeakyReLU · Tanh/Sigmoid

Generator — z → Görüntü

`z (100,)`	Gürültü vektörü
↓	
`Dense → Reshape`	→ (4, 4, 256)
↓	
`ConvT2D 128, 4×4`	BatchNorm + ReLU → 8×8
↓	
`ConvT2D 64, 4×4`	BatchNorm + ReLU → 16×16
↓	
`ConvT2D 32, 4×4`	BatchNorm + ReLU → 32×32
↓	
`ConvT2D 1, 4×4`	Tanh → 64×64×1 ✓

Discriminator — Görüntü → Skor

`Girdi (64×64×1)`	Gerçek veya sahte
↓	
`Conv2D 32, 4×4`	LeakyReLU(0.2) → 32×32
↓	
`Conv2D 64, 4×4`	BN + LeakyReLU → 16×16
↓	
`Conv2D 128, 4×4`	BN + LeakyReLU → 8×8
↓	
`Conv2D 256, 4×4`	BN + LeakyReLU → 4×4
↓	
`Flatten → Dense(1)`	Sigmoid → P(gerçek) ✓

DCGAN kuralları: Pooling yok (stride Conv2D) · FC yok (sadece son katmanda) · BatchNorm · ReLU (G), LeakyReLU (D), Tanh (G son)

DCGAN — Keras ile Fashion-MNIST Uygulaması

Eğitim Döngüsü · Adversarial Loss · Görüntü Üretimi · Eğitim Takibi

dcgan_fashion_mnist.py

```
import tensorflow as tf
from tensorflow import keras
import numpy as np, os

# ── Veri ──────────────────────────────────────────
(x_train,_),(_,_) = keras.datasets.fashion_mnist.load_data()
x_train = (x_train.astype('float32') - 127.5) / 127.5  # [-1,1]
x_train = x_train[..., np.newaxis]   # (60000,28,28,1)

# ── GAN sınıfı ────────────────────────────────────
class GAN(keras.Model):
    def __init__(self, discriminator, generator, latent_dim):
        super().__init__()
        self.discriminator = discriminator
        self.generator     = generator
        self.latent_dim    = latent_dim

    def compile(self, d_optimizer, g_optimizer, loss_fn):
        super().compile()
        self.d_opt, self.g_opt = d_optimizer, g_optimizer
        self.loss_fn = loss_fn

    def train_step(self, real_images):
        batch_size = tf.shape(real_images)[0]
        z = tf.random.normal([batch_size, self.latent_dim])
        with tf.GradientTape() as tape:  # D güncelle
            real_logits = self.discriminator(real_images, training=True)
```

dcgan_train.py

```
gan = GAN(discriminator, generator, latent_dim=100)
gan.compile(
    d_optimizer=keras.optimizers.Adam(2e-4, beta_1=0.5),
    g_optimizer=keras.optimizers.Adam(2e-4, beta_1=0.5),
    loss_fn=keras.losses.BinaryCrossentropy(from_logits=True)
)
gan.fit(x_train, epochs=50, batch_size=64,
        callbacks=[GANMonitor(n_samples=16, latent_dim=100)])
```

β_1=0.5 (Adam)

Standart β_1=0.9 yerine 0.5 → GAN eğitimi daha kararlı.

Label Smoothing

Gerçek etiket: 1.0→0.9. Discriminator'ın aşırı güvenmesini önler.

GANMonitor

Her epoch sonunda callback ile 16 sahte görüntü kaydeder, ilerlemeyi takip eder.

Fashion-MNIST: 28×28 gri kıyafet görüntüleri (t-shirt, pantolon, elbise...) · 60K train · Beklenen süre: ~10 dk GPU / ~45 dk CPU

VAE vs GAN vs Difüzyon — Karşılaştırma

Görüntü Kalitesi · Eğitim Kararlılığı · Latent Space · Kullanım Alanı

Özellik	VAE	GAN / DCGAN	Difüzyon
Yaklaşım	Encoder-Decoder latent z öğrenir	Generator+Disc. Adversarial oyun	İleri+geri gürültü iteratif denoise
Kayıp	ELBO = BCE + KL	BCE (minimax)	MSE (noise pred)
Görüntü Kalite	Orta (bulanık)	Yüksek (keskin)	Çok Yüksek SOTA
Eğitim Kararlılık	Kararlı ✓	Hassas mode coll.	Çok kararlı ✓
Latent Yapısı	Yapılandırılmış ✓	Yapısız	Gizli (steerable)
Hız	Hızlı üretim	Hızlı üretim	Yavaş ~1000 adım
Kullanım	Anomali tespiti Veri artırma	Yüz / sahne üretimi	Text→image Stable Diffusion

Seçim rehberi: Latent analiz → VAE · Keskin görüntü üretimi → GAN · En yüksek kalite → Difüzyon · Hafta 5: Stable Diffusion

Bugünkü Uygulamalar — 4 Python Dosyası

pip install tensorflow keras numpy matplotlib scikit-learn

Uygulama 01 — VAE: MNIST Latent Space

`uygulama_01_vae_mnist.py`

- Encoder/Decoder Keras ile sıfırdan inşa
- Reparameterization Layer (özel katman)
- ELBO kaybı: Reconstruction + KL bileşenleri
- 2D Latent Space görselleştirme (10 sınıf)
- 20×20 manifold grid: rakam geçişleri

Uygulama 02 — VAE: KL Annealing & Analiz

`uygulama_02_vae_analiz.py`

- β-VAE ile KL ağırlık ablasyonu (β=0.1~5)
- KL annealing: giderek artan β stratejisi
- Latent boyut ablasyonu: z_dim=2,4,8,16
- Interpolasyon: iki rakam arasında geçiş
- FID skoru simülasyonu

Uygulama 03 — DCGAN: Fashion-MNIST

`uygulama_03_dcgan.py`

- ConvTranspose2D Generator mimarisi
- Strided Conv2D Discriminator
- GANMonitor callback: epoch görüntüleri
- Eğitim kaybı: d_loss vs g_loss grafikleri
- Sonuç: 8×8 grid üretilmiş kıyafet görüntüsü

Uygulama 04 — GAN Değerlendirme & Sorunlar

`uygulama_04_gan_degerlendirme.py`

- FID (Fréchet Inception Distance) hesaplama
- Mode collapse tespiti ve önleme stratejileri
- Discriminator/Generator denge analizi
- Inception Score (IS) hesaplama
- VAE vs GAN görsel kalite karşılaştırması

```
pip install tensorflow keras numpy matplotlib scikit-learn  |  GPU opsiyonel — CPU ile de çalışır (latent_dim=2, epoch=10-30)
```

Hafta 4 Pazar — Kazanımlar

- ✓ Generatif vs diskriminatif ayrımı: $P(y|x)$ vs $P(x)$
- ✓ Reparameterization trick: $z = \mu + \sigma \cdot \varepsilon$ (gradyan akar)
- ✓ GAN minimax oyunu: min G max D $V(D,G)$
- ✓ Adversarial eğitim döngüsü: D güncelle → G güncelle
- ✓ VAE matematiği: ELBO = Reconstruction + KL
- ✓ 2D latent space görselleştirme: küme + manifold
- ✓ DCGAN kuralları: ConvTranspose, BatchNorm, LeakyReLU
- ✓ VAE vs GAN vs Difüzyon: ne zaman hangisi seçilir?

📅 **Hafta 5 Cumartesi: Stable Diffusion · LLM Prompt Mühendisliği · Flask/FastAPI ile Model Deployment**